Unit 4 – Blockchain and Decentralized Applications

Lab 4 Manual

Introduction to Solidity Programming with VS Code and Truffle

Objective:

In this lab, students will learn, implement and integrate the real-world use case of mappings, structs, and functions to create more complex data management structures, enabling scalable and efficient data handling.

By the end of this lab, students will:

- Understand mappings and how they store key-value pairs in Solidity.
- Learn how structs are used to group related variables.
- Explore combining mappings with structs for more complex data management.

A. Create a contract to store and retrieve messages, showcasing memory vs. storage usage.

```
pragma solidity ^0.8.0;

UnitTest stub | dependencies | uml | funcSigs | draw.io
contract Messages {
    struct MessageContainer {
        string message;
    }

    mapping(string => MessageContainer) public messages;

    ftrace | funcSig
    function setMessage(string memory msgContent) public {
        messages[msgContent] = MessageContainer(msgContent);
    }

    ftrace | funcSig
    function getMessage(string memory msgContent) public view returns (MessageContainer memory) {
        return messages[msgContent];
    }
}
```

```
> await messages.setMessage("Hello, Solidityyyyy!");
ContractTransactionResponse {
  provider: HardhatEthersProvider {},
  blockNumber: 11,
  blockHash: '0x929e30623e2a929d2f218692baf5b8a0bafec80a85ea6a63c5e99cd97e807224',
  index: undefined,
  hash: '0xaf2c9429f5ec79e8cec04aeb932413eaf097863f713b7fe3f0c048caf8e16981',
  type: 2,
  to: '0x2279B7A0a67DB372996a5FaB50D91eAA73d2eBe6',
  from: '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266',
  nonce: 10,
  gasLimit: 45794n,
  gasPrice: 295941239n,
  maxPriorityFeePerGas: 70968682n,
  maxFeePerGas: 295941239n,
  maxFeePerBlobGas: null,
  data: '0x368b8772000000000000000000000000000000000000000000000000000000000000002000000000000000000000000000000000000000000000000000000000000000001448656c6c6f2c20536f6c6964697479797979792100000000000000000000000000',
  value: 0n,
  chainId: 31337n,
  signature: Signature { r: 0xa371d95606e6984b1d02d4a89c1aefebc9db62ef1cb5abf576fbc0611a124a31, s: 0x4262d54c13d004ec07d24bdadaff6aafc71cfcacf6e98f89fd63313a7c597eb9, v: 28 },
  accessList: [],
  blobVersionedHashes: null,
  authorizationList: null
}
>
> await messages.getMessage("Hello, Solidity!");
Proxy [ Result(1) [ 'Hello, Solidity!' ], { get: [Function: get] } ]
> await messages.getMessage("Hello, Solidityyy!");
Proxy [ Result(1) [ '' ], { get: [Function: get] } ]
> await messages.getMessage("Hello, Solidityy!");
Proxy [ Result(1) [ '' ], { get: [Function: get] } ]
> await messages.getMessage("Hello, Solidityyyyy!");
Proxy [
  Result(1) [ 'Hello, Solidityyyyy!' ],
  { get: [Function: get] }
]
```

B. Develop a contract to represent a person's information and retrieve it.

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

UnitTest stub | dependencies | uml | funcSigs | draw.io
contract PersonInfo {
    struct Person {
        string name;
        uint256 age;
    }

    Person public person;

    ftrace | funcSig
    function setPerson(string memory name↑, uint256 age↑) public {
        person = Person(name↑, age↑);
    }

    ftrace | funcSig
    function getPerson() public view returns (Person memory) {
        return person;
    }
}
```

```
gabs@gabs-beast:~/Documenti/Università/Software Security & Blockchain/Esercitazioni/blocky-the-blockchain$ npm run console

> lab2-hardhat-2026@1.0.0 console
> hardhat console --network localhost

> const { ethers } = await network.connect()
undefined
> F = await ethers.getContractFactory("PersonInfo") //PersonInfo.sol
ContractFactory {
  interface: Interface {
    fragments: [ [FunctionFragment], [FunctionFragment], [FunctionFragment] ],
    deploy: ConstructorFragment {
      type: 'constructor',
      inputs: [],
      payable: false,
      gas: null
    },
    fallback: null,
    receive: false
  },
  bytecode: '0x6080604052348015600e575f5ffd5b506108628061001c5f395ff3fe608060405234801561000f575f5ffd5b5060043610610 [illegible] 8020188555050505b50505050505056fea2646970667358221220e8db9e2beb4ca04e34f70d35ffab641851497979f24ebd [illegible]',
  runner: HardhatEthersSigner {
    address: '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266',
    provider: HardhatEthersProvider {}
  }
}
> c = await F.deploy()
Proxy [
  BaseContract {
    target: '0xCf7Ed3AccA5a467e9e704C703E8D87F634fB0Fc9',
    interface: Interface {
      fragments: [Array],
      deploy: [ConstructorFragment],
      fallback: null,
      receive: false
    },
    runner: HardhatEthersSigner {
      address: '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266',
      provider: HardhatEthersProvider {}
    },
    filters: Proxy [ {}, [Object] ],
    fallback: null,
    [Symbol(_ethersInternal_contract)]: {}
  },
  { get: [Function: get], has: [Function: has] }
]
```

```
> await c.waitforDeployment()
Uncaught TypeError: c.waitforDeployment is not a function
    at REPL64:1:41
> await c.waitForDeployment()
Proxy [
  BaseContract {
    target: '0xCf7Ed3AccA5a467e9e704C703E8D87F634fB0Fc9',
    interface: Interface {
      fragments: [Array],
      deploy: [ConstructorFragment],
      fallback: null,
      receive: false
    },
    runner: HardhatEthersSigner {
      address: '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266',
      provider: HardhatEthersProvider {}
    },
    filters: Proxy [ {}, [Object] ],
    fallback: null,
    [Symbol(_ethersInternal_contract)]: {}
  },
  { get: [Function: get], has: [Function: has] }
]
> await (await c.setPerson("Sergio",29)).wait()
ContractTransactionReceipt {
  provider: HardhatEthersProvider {},
  to: '0xCf7Ed3AccA5a467e9e704C703E8D87F634fB0Fc9',
  from: '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266',
  contractAddress: null,
  hash: '0x530a06a479ac4df5c0cd98d50179dafe563bbf802ae07a4c99e3df2416b8ceab',
  index: 0,
  blockHash: '0x63ddeed89f9f6c2269c93ca971d7b9a77013ac6b838ddd21a23b19fc16be524a',
  blockNumber: 5,
  logsBloom: '0x00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000',
  gasUsed: 67673n,
  blobGasUsed: undefined,
  cumulativeGasUsed: 67673n,
  gasPrice: 652778183n,
  blobGasPrice: undefined,
  type: 2,
  status: 1,
  root: undefined
}
> await c.getPerson()
Proxy [ Result(2) [ 'Sergio', 29n ], { get: [Function: get] } ]
>
```

C. Demonstrate how to use an array of structs to manage multiple records.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

UnitTest stub | dependencies | uml | funcSigs | draw.io
contract StructArray {
    struct Person {
        string name;
        string eventt;
    }

    Person[] public persons;

    ftrace | funcSig
    function addPerson(string memory name, string memory eventt) public {
        persons.push(Person(name, eventt));
    }
}
```

```
gabs@gabs-beast:~/Documenti/Università/Software Security & Blockchain/Esercitazioni/blocky-the-blockchain$ npm run console

> lab2-hardhat-2026@1.0.0 console
> hardhat console --network localhost

> const { ethers } = await network.connect()
undefined
> let F = await ethers.getContractFactory("StructArray")
undefined
> let c = await F.deploy()
undefined
> await c.waitForDeployment()
Proxy [
  BaseContract {
    target: '0x5FbDB2315678afecb367f032d93F642f64180aa3',
    interface: Interface {
      fragments: [Array],
      deploy: [ConstructorFragment],
      fallback: null,
      receive: false
    },
    runner: HardhatEthersSigner {
      address: '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266',
      provider: HardhatEthersProvider {}
    },
    filters: Proxy [ {}, [Object] ],
    fallback: null,
    [Symbol(_ethersInternal_contract)]: {}
  },
  { get: [Function: get], has: [Function: has] }
]
>
> await (await c.addPerson("Mario", "Blockchain")).wait()
ContractTransactionReceipt {
  provider: HardhatEthersProvider {},
  to: '0x5FbDB2315678afecb367f032d93F642f64180aa3',
  from: '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266',
  contractAddress: null,
  hash: '0x58652f1fcdc99dc191ce7ce05bdbe6853d0aa4d84d1efb719759da4eeecb07c0',
  index: 0,
  blockHash: '0x752bfd6c64a5014ff0c8f98ec18bb650a96e111c7ecb0da3628e52c26ddbeb8b',
  blockNumber: 2,
  logsBloom: '0x00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000',
  gasUsed: 91403n,
  blobGasUsed: undefined,
  cumulativeGasUsed: 91403n,
  gasPrice: 971251366n,
  blobGasPrice: undefined,
  type: 2,
  status: 1,
  root: undefined
}
> await c.persons(0)
Proxy [ Result(2) [ 'Mario', 'Blockchain' ], { get: [Function: get] } ]
> await (await c.addPerson("Paolino", "Blockchain")).wait()
ContractTransactionReceipt {
  provider: HardhatEthersProvider {},
  to: '0x5FbDB2315678afecb367f032d93F642f64180aa3',
  from: '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266',
  contractAddress: null,
  hash: '0x06493c3433d6d9a6c8e56d089702eae413716b8e2ce28d3e26f5a6e2a065b752',
  index: 0,
  blockHash: '0xba2938f8b664a90114fb647a873a370daa6028b7f8233b3e250385381564020f',
  blockNumber: 3,
  logsBloom: '0x00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000',
  gasUsed: 74327n,
  blobGasUsed: undefined,
  cumulativeGasUsed: 74327n,
  gasPrice: 850214843n,
  blobGasPrice: undefined,
  type: 2,
```

```
gabs@gabs-beast:~/Documenti/Università/Software Security & Blockchain/Esercitazioni/blocky-the-blockchain$ npm run console
  blockHash: '0x752bfd6c64a5014ff0c8f98ec18bb650a96e111c7ecb0da3628e52c26ddbeb8b',
  blockNumber: 2,
  logsBloom: '0x00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000',
  gasUsed: 91403n,
  blobGasUsed: undefined,
  cumulativeGasUsed: 91403n,
  gasPrice: 971251366n,
  blobGasPrice: undefined,
  type: 2,
  status: 1,
  root: undefined
}
> await c.persons(0)
Proxy [ Result(2) [ 'Mario', 'Blockchain' ], { get: [Function: get] } ]
> await (await c.addPerson("Paolino", "Blockchain")).wait()
ContractTransactionReceipt {
  provider: HardhatEthersProvider {},
  to: '0x5FbDB2315678afecb367f032d93F642f64180aa3',
  from: '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266',
  contractAddress: null,
  hash: '0x06493c3433d6d9a6c8e56d089702eae413716b8e2ce28d3e26f5a6e2a065b752',
  index: 0,
  blockHash: '0xba2938f8b664a90114fb647a873a370daa6028b7f8233b3e250385381564020f',
  blockNumber: 3,
  logsBloom: '0x00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000',
  gasUsed: 74327n,
  blobGasUsed: undefined,
  cumulativeGasUsed: 74327n,
  gasPrice: 850214843n,
  blobGasPrice: undefined,
  type: 2,
  status: 1,
  root: undefined
}
> await c.persons
[AsyncFunction: persons] {
  name: 'persons',
  _contract: BaseContract {
    target: '0x5FbDB2315678afecb367f032d93F642f64180aa3',
    interface: Interface {
      fragments: [Array],
      deploy: [ConstructorFragment],
      fallback: null,
      receive: false
    },
    runner: HardhatEthersSigner {
      address: '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266',
      provider: HardhatEthersProvider {}
    },
    filters: Proxy [ {}, [Object] ],
    fallback: null,
    [Symbol(_ethersInternal_contract)]: {}
  },
  _key: 'persons',
  getFragment: [Function: getFragment],
  estimateGas: [AsyncFunction: estimateGas],
  populateTransaction: [AsyncFunction: populateTransaction],
  send: [AsyncFunction: send],
  staticCall: [AsyncFunction: staticCall],
  staticCallResult: [AsyncFunction: staticCallResult],
  fragment: [Getter]
}
> await c.persons(0)
Proxy [ Result(2) [ 'Mario', 'Blockchain' ], { get: [Function: get] } ]
> await c.persons(1)
Proxy [
  Result(2) [ 'Paolino', 'Blockchain' ],
  { get: [Function: get] }
]
> 
```

D. Create and manage nested structures using a contract.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

UnitTest stub | dependencies | uml | funcSigs | draw.io
contract NestedStruct {
    struct Address {
        uint streetNo;
        string city;
    }

    struct Student {
        string name;
        bool status;
        Address addressDetails;
    }

    Student[] public students;

    ftrace | funcSig
    function addStudent(string memory name, bool status, uint streetNo, string memory city) public {
        Address memory addr = Address(streetNo, city);
        students.push(Student(name, status, addr));
    }

    ftrace | funcSig
    function getStudent(uint index) public view returns (Student memory) {
        return students[index];
    }
}
```

```
gabs@gabs-beast:~/Documenti/Università/Software Security & Blockchain/Esercitazioni/blocky-the-blockchain$ npm run console

> lab2-hardhat-2026@1.0.0 console
> hardhat console --network localhost

> const { ethers } = await network.connect()
undefined
> F = await ethers.getContractFactory("NestedStruct")
ContractFactory {
  interface: Interface {
    fragments: [ [FunctionFragment], [FunctionFragment], [FunctionFragment] ],
    deploy: ConstructorFragment {
      type: 'constructor',
      inputs: [],
      payable: false,
      gas: null
    },
    fallback: null,
    receive: false
  },
  bytecode: '0x6080604052348015600e575f5ffd5b50610c668061001c5f395ff3fe608060405234801561000f575f5ffd5b506004361061003f575f3560e01c806306ead22e1461004357806
0915775ffd5b61005d6004803603810190610058919061004ff565b6100c15656b6040516100c939291906106455656b6040518091039
06100a691906104ff565b6102eb565b60405161006b91906101917565b6040518091039
040519081016040528092919081815260200182805461041a906106456565b80156101655780601f1061013c576101008083540402835291602001916101655656b8201919
29003601f168201915b5050505050908060001015f9054906101000a900460ff16908060020160405180604001604052908152602001600182018054601a5906106456
092919081815260200182805461d190610964565b801561021c5780601f106101f3576101008083540402835291602001916102
b505050505081525050905083565b5f6040518060400160405280848152602001838152509050560400201861515815260200183815250908060001815401808255809150506001900
402015f9091909190915051f820151815f01908161029991906010b34565b506020820151816001015f6101000a81548160ff0219169083151502179055506040820151816002015f820151815f015560208201518160010190816102df9190610
b34565b5050505050505050505050565b6102f361047a565b5f82815481106103065761030561 0c03565b5b905f5260205f2090600402016040518060600160405290815f8201805461032c9061096456565b80601f0160208091040260200160405
190810160405280929190818152602001828054610358906109645656b80156103a5780601f1061037a576101008083540402835291602001916103a3565b820191905f5260205f20905b81548152906001019060200180831161038657829003601f168201915b5050505050815260200160018205f9054906101000a900460ff16151515158152602001600282016040518060400160405290815f8201548152602001600182018054610 3ef90610964565b80601f0160208091040260200160405190810160405280929190818152602001828054610389060109645656b80156103d5780601f106103ab576101008083540402835291602001916103d5565b820191905f5260205f20905b81548152906001019060200180831161049578290036 01f168201915b5050505050815250508152050905091905056 5b6040518060400160405280600815260200015f1515815260200161049c6104a2565b81525090565b60405180604001604052805f81526020016006815209056 5b5f6040519050905656b5f5ffd5b5f5ffd5b5f81905091905056 5b6104de816104cc565b81146104e8575f5ffd5b50565b5f81359050610 4f9816104d5565b92915050565b5f6020828403121561051457610513610 4c4565b5b5f6105218482850 16104eb565b91505092915050565b5f81519050919050565b5f82825260208201905092915050565b5f61056c8261052a565b61057681856105345 65b6105e78185610 5c3565b93506105f781856020860161054456 5b6100816105525 65b840191505092915050565b5f6040830 15f830151610620 5f8601826105b4565b506020830151848203602086015261063882826105d3565b9150508 0915050929150505 65b5f6060820190508181035f83015261065d8186105625 65b905061066c6020830185105a5565b8181036040830151261067e818461060b565b905094935050505065b5f5ffd5b5f5ffd5b7f4e487b7100000000000000
0000000000000000000000000000000000000000000000000000005f526041604526024 5ffd5b6106c6826105525 65b810181811067fffffffffffffffff821117156106e5576106e4610690565b5b8060405250505056 5b5f6106f76104bb565b905061070
382826106d565b91905056 5b5f67ffffffffffffffff8211156107225761072161069056 5b5b1072b826105525 65b905060208101905091905056 5b828183375f8383015250505056 5b5f610758610753846107085656b6106ee565b9050828
152602081018484840111156107745761077361068c565b5b61077f6482856107385 65b5093925050565b5f82601f83011261079b5761079a610685 65b8135610 7ab84826020860161074656 5b9150509215050565b6107bd8161059a5
65b81146107c7575f5ffd5b50565b5f813590506107d8816107b4565b9291505056 5b5f5f5f6080858703121561076f6576107f56104c4565b5b5f85013567ffffffffffffffff8111156108137561081261 04c4565b5b61081f87828801610
787565b9450506020610830878288016107ca565b935050604061084187828801610 4eb565b9250506060850135676ffffffffffffff8111156108257610861 06e565b5b61086e878288016107875656b9150509295919450925056 5b610
883816105 9a565b8252505056 5b5f604083015f830151610 89e5f8601826105b4565b506020830151848203602086015261 08b682826105d3565b9150508091505092915050565b5f606083015f830151848203 5f8601526108dd82826105d35
65b9150506020830151610 8f26020860182610 87a565b506040830151848203604086015261090a828261088956 5b9150508091505092915050565b5f602082019050818103 5f830152610 92f81846108c3565b905092915050565b7f4e487b7
1000000000000000000000000000000000000000000000000000005f526022604526024 5ffd5b5f6002820495060018216806109 7b57607f821691505b602082108103610 98e5761098d610937565b5b5091905056 5b5f819050815f526
0205f2090509190505656b5f6020601f830104905091905056 5b5f82821b905092915050565b5f6008830261090f07fffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff826109b5565b6109fa86836109b55656b955
08019841693508086168417925050509392505056 5b5f8190509190505656b5f610a35610a30610a2b846104cc565b610a12565b6104cc565b905091905056 5b5f819050919050565b610a4e83610a1b565b610a62610a5a82610a3c565b848
4546109c1565b825550505050565b5f5f905090565b610a79610a6a565b610a84818484610a45565b505050565b5b5b8181015610aa757610a9c5f82610a71565b60018101905061 0a8a565b5050565b601f821115610aec57610abd816109945
65b610ac6846109a6565b8101602085101561 0ad5578190505b610ae9610ae1856109a6565b830182610a89565b50505b505050565b5f82821c905092915050565b5f610b0c5f1984600802610af1565b1980831691505092915050565b5f610
b248383610afd565b91508260020282179050929150505656b610b3d8261052a565b67ffffffffffffffff811115610b5657610969565b5b610b6825461096456 5b610b828285610aab565b5f602090506 01f831160018114610b9c5
75f84156 10b8a5782870151905056 5b610b948582610b19565b865556 10bfb565b601f198416610baa866109945656 5b5f5b82811015610bd157848901518255560018201915060208501945060208101905061 0bac565b86831015610bee5784890
151610bea601f891682610afd565b8355505b6001602882021885555050505b505050505050565b7f4e487b7100000000000000000000000000000000000000000000000000000000005f526032600452602 45ffdfea2646970667358221204e7
334e04d4e5724 41bfdd30fd987d764aad1ddaa87846 39a1eadd267525a5064736f6c634300081c0033',
  runner: HardhatEthersSigner {
    address: '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266',
    provider: HardhatEthersProvider {}
  }
}
> c = await F.deploy()
Proxy [
  BaseContract {
    target: '0x5FC8d32690cc91D4c39d9d3abcBD16989F875707',
    interface: Interface {
      fragments: [Array],
      deploy: [ConstructorFragment],
      fallback: null,
      receive: false
    },
    runner: HardhatEthersSigner {
      address: '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266',
      provider: HardhatEthersProvider {}
    },
    filters: Proxy [ {}, [Object] ],
    fallback: null,
```

```
> await c.getStudent(0)
Proxy [
  Result(3) [ 'Mario', true, Proxy [ [Result], [Object] ] ],
  { get: [Function: get] }
]
> await (await c.addStudent("Marcolino", true, 1343, "Ancona")).wait()
ContractTransactionReceipt {
  provider: HardhatEthersProvider {},
  to: '0x5FC8d32690cc91D4c39d9d3abcBD16989F875707',
  from: '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266',
  contractAddress: null,
  hash: '0xe1885f30dea0da5b922cb79a2ae99645d856463686 65ec0a8265e9f4c67392d1',
  index: 0,
  blockHash: '0xbbb8599b2a5b0354f13dbe4d67f8026e3107ff7abdbecfc5cbe5f38724151758',
  blockNumber: 8,
  logsBloom: '0x00000000000000000000000000000000000000000000000000000000
```

E. Combine nested structures with mappings to manage data efficiently.

Blockchain_Decentralized_Apps / Lab4 / Lab 4_E.sol

S1120957 renamed

Code | Blame | 27 lines (23 loc) · 581 Bytes

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract NestedStructMapping {
    struct Address {
        uint streetNo;
        string city;
    }

    struct Student {
        string name;
        bool status;
        Address addressDetails;
    }

    mapping(uint => Student) public studentRecords;

    function addStudent(
        uint id,
        string memory name,
        bool status,
        uint streetNo,
        string memory city
    ) public {
        studentRecords[id] = Student(name, status, Address(streetNo, city));
    }
}
```

```
> const { ethers } = await network.connect()
undefined
> F = await ethers.getContractFactory("NestedStructMapping")
ContractFactory {
  interface: Interface {
    fragments: [ [FunctionFragment], [FunctionFragment] ],
    deploy: ConstructorFragment {
      type: 'constructor',
      inputs: [],
      payable: false,
      gas: null
    },
    fallback: null,
    receive: false
  },
  bytecode: '0x6080604052348015600e575f5ffd5b5061095b8061001c5f395ff3fe608060405234801561000f575f5ffd5b506004361061003457
...
  runner: HardhatEthersSigner {
    address: '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266',
    provider: HardhatEthersProvider {}
  }
}
> c = await F.deploy()
Proxy [
  BaseContract {
    target: '0x2279B7A0a67DB372996a5FaB50D91eAA73d2eBe6',
    interface: Interface {
      fragments: [Array],
      deploy: [ConstructorFragment],
      fallback: null,
      receive: false
    },
    runner: HardhatEthersSigner {
      address: '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266',
      provider: HardhatEthersProvider {}
    },
    filters: Proxy [ {}, [Object] ],
    fallback: null,
    [Symbol(_ethersInternal_contract)]: {}
  },
  { get: [Function: get], has: [Function: has] }
]
> await c.waitForDeployment()
Proxy [
  BaseContract {
    target: '0x2279B7A0a67DB372996a5FaB50D91eAA73d2eBe6',
```

```
    filters: Proxy [ {}, [Object] ],
    fallback: null,
    [Symbol(_ethersInternal_contract)]: {}
  },
  { get: [Function: get], has: [Function: has] }
]
> await c.waitForDeployment()
Proxy [
  BaseContract {
    target: '0x2279B7A0a67DB372996a5FaB50D91eAA73d2eBe6',
    interface: Interface {
      fragments: [Array],
      deploy: [ConstructorFragment],
      fallback: null,
      receive: false
    },
    runner: HardhatEthersSigner {
      address: '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266',
      provider: HardhatEthersProvider {}
    },
    filters: Proxy [ {}, [Object] ],
    fallback: null,
    [Symbol(_ethersInternal_contract)]: {}
  },
  { get: [Function: get], has: [Function: has] }
]
>
> await (await c.addStudent(1, "Mario", true, 10, "Roma")).wait()
ContractTransactionReceipt {
  provider: HardhatEthersProvider {},
  to: '0x2279B7A0a67DB372996a5FaB50D91eAA73d2eBe6',
  from: '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266',
  contractAddress: null,
  hash: '0xc95ae4bb05aea998a4299b718405344b95bb57331e29e6223765b985f1862c5f',
  index: 0,
  blockHash: '0x6d7532a16282856050d21abee27162b032702e234898e4efea41545227b166f5',
  blockNumber: 10,
  logsBloom: '0x00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000',
  gasUsed: 114523n,
  blobGasUsed: undefined,
  cumulativeGasUsed: 114523n,
  gasPrice: 337425264n,
  blobGasPrice: undefined,
  type: 2,
  status: 1,
  root: undefined
}
> await c.studentRecords(1)
Proxy [
  Result(3) [ 'Mario', true, Proxy [ [Result], [Object] ] ],
  { get: [Function: get] }
]
```

Lab Tasks

1. Simulate a ticket booking system using mappings and structs. Also, use *require, assert and revert* for error handling.

Code:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

contract Ticket{
    struct TicketContainer{
        uint256 id;
        address owner;
        uint256 price;
        bool valid;
    }

    mapping(uint256 => TicketContainer) tickets;

    function initTicket (uint256 ticketnumber, uint256 money) external
    {
        require(ticketnumber !=0, "it's a invalid ticketnumber!");

        if(tickets[ticketnumber].valid)
        {
            revert("Ticket already exist mate!");
        }

        // require(!tickets[ticketnumber].valid, "Ticket already exist mate!");   //old code

        tickets[ticketnumber]=TicketContainer({
            id: ticketnumber,
            owner: msg.sender,
            price: money,
            valid: true
        });

        // check invariant state
        assert(tickets[ticketnumber].valid);
        assert(tickets[ticketnumber].owner==msg.sender);
    }

    function getTicket(uint256 ticketnumber) external view returns (TicketContainer memory)
    {
        require(tickets[ticketnumber].valid, "ticka must be valid mate!");
        return tickets[ticketnumber];
    }

    function invalidateTicket(uint256 ticketnumber) external
    {
        require(tickets[ticketnumber].valid, "ticket not valid!");

        if (tickets[ticketnumber].owner!=msg.sender)
        {
            revert("I don't think you own this ticket mate!");
        }

        tickets[ticketnumber].valid=false;

        // check invariant state
        assert(!tickets[ticketnumber].valid);
    }

}
```

Result:

```
> const tick = await ethers.deployContract("Ticket")
undefined
> await tick.waitForDeployment()
Proxy [
  BaseContract {
    target: '0x5FbDB2315678afecb367f032d93F642f64180aa3',
    interface: Interface {
      fragments: [Array],
      deploy: [ConstructorFragment],
      fallback: null,
      receive: false
    },
    runner: HardhatEthersSigner {
      address: '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266',
      provider: HardhatEthersProvider {}
    },
    filters: Proxy [ {}, [Object] ],
    fallback: null,
    [Symbol(_ethersInternal_contract)]: {}
  },
  { get: [Function: get], has: [Function: has] }
]
> tick.getAddress()
Promise {
  <pending>,
  [Symbol(async_id_symbol)]: 22713,
  [Symbol(trigger_async_id_symbol)]: 403
}
> await (await tick.initTicket(1,100)).wait()
ContractTransactionReceipt {
  provider: HardhatEthersProvider {},
  to: '0x5FbDB2315678afecb367f032d93F642f64180aa3',
  from: '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266',
  contractAddress: null,
  hash: '0xc1da6cae2ab4d3fb2d3dd76a3fab7cdcafb322abaaa59cfee18a627dc2963ca5',
  index: 0,
  blockHash: '0xd83bcf423150c682146ceb3fcd56e3c70871bb21416a6a8cc3981601c593ccfd',
  blockNumber: 2,
  logsBloom: '0x00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000',
  gasUsed: 111427n,
  blobGasUsed: undefined,
  cumulativeGasUsed: 111427n,
  gasPrice: 971627525n,
  blobGasPrice: undefined,
  type: 2,
  status: 1,
  root: undefined
}
> await (await tick.initTicket(1,300)).wait()
Uncaught:
ProviderError: VM Exception while processing transaction: reverted with reason string 'Ticket already exist mate!'
    at HttpProvider.request (/home/gabs/Documenti/Università/Software Security & Blockchain/Esercitazioni/blocky-the-blockchain/node_modules/hardhat/src/internal/builtin-plugins/network-manager/http-provider.ts:144:21)
    at process.processTicksAndRejections (node:internal/process/task_queues:103:5)
    at async AutomaticGasHandler.getMultipliedGasEstimation (/home/gabs/Documenti/Università/Software Security & Blockchain/Esercitazioni/blocky-the-blockchain/node_modules/hardhat/src/internal/builtin-plugins/network-manager/request-handlers/handlers/gas/multiplied-gas-estimation.ts:50:30)
    at async AutomaticGasHandler.handle (/home/gabs/Documenti/Università/Software Security & Blockchain/Esercitazioni/blocky-the-blockchain/node_modules/hardhat/src/internal/builtin-plugins/network-manager/request-handlers/handlers/gas/automatic-gas-handler.ts:54:16)
    at async Array.onRequest (/home/gabs/Documenti/Università/Software Security & Blockchain/Esercitazioni/blocky-the-blockchain/node_modules/hardhat/src/internal/builtin-plugins/network-manager/hook-handlers/network.ts:72:38)
    at async next (/home/gabs/Documenti/Università/Software Security & Blockchain/Esercitazioni/blocky-the-blockchain/node_modules/hardhat/src/internal/core/hook-manager.ts:207:13)
    at async HookManagerImplementation.runHandlerChain (/home/gabs/Documenti/Università/Software Security & Blockchain/Esercitazioni/blocky-the-blockchain/node_modules/hardhat/src/internal/core/hook-manager.ts:213:12)
    at async HttpProvider.request (/home/gabs/Documenti/Università/Software Security & Blockchain/Esercitazioni/blocky-the-blockchain/node_modules/hardhat/src/internal/builtin-plugins/network-manager/http-provider.ts:135:25)
    at async HardhatEthersProvider.send (/home/gabs/Documenti/Università/Software Security & Blockchain/Esercitazioni/blocky-the-blockchain/node_modules/@nomicfoundation/hardhat-ethers/src/internal/hardhat-ethers-provider/hardhat-ethers-provider.ts:119:12) {
  code: 3,
  data: '0x08c379a00000000000000000000000000000000000000000000000000000000000000020000000000000000000000000000000000000000000000000000000000000001a5469636b657420616c7265616479206578697374206d6174652100000000000000'
}
```

```
> await (await tick.invalidateTicket(1)).wait()
ContractTransactionReceipt {
  provider: HardhatEthersProvider {},
  to: '0x5FbDB2315678afecb367f032d93F642f64180aa3',
  from: '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266',
  contractAddress: null,
  hash: '0x5b4a5e24251735813480cabfe63185ae063f7ea959113e9a6023850185db5659',
  index: 0,
  blockHash: '0x9589a14ec47196494b29ddd13eb85951501a15ee59c54e206b1c020367ca763c',
  blockNumber: 3,
  logsBloom: '0x00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000',
  gasUsed: 24692n,
  blobGasUsed: undefined,
  cumulativeGasUsed: 24692n,
  gasPrice: 850625191n,
  blobGasPrice: undefined,
  type: 2,
  status: 1,
  root: undefined
}
> await tick.getTicket(1)
Uncaught:
ProviderError: VM Exception while processing transaction: reverted with reason string 'ticka must be valid mate!'
    at HttpProvider.request (/home/gabs/Documenti/Università/Software Security & Blockchain/Esercitazioni/blocky-the-blockchain/node_modules/hardhat/src/internal/builtin-plugins/network-manager/http-provider.ts:144:21)
    at process.processTicksAndRejections (node:internal/process/task_queues:103:5)
    at async HardhatEthersProvider.call (/home/gabs/Documenti/Università/Software Security & Blockchain/Esercitazioni/blocky-the-blockchain/node_modules/@nomicfoundation/hardhat-ethers/src/internal/hardhat-ethers-provider/hardhat-ethers-provider.ts:333:18)
    at async HardhatEthersSigner.call (/home/gabs/Documenti/Università/Software Security & Blockchain/Esercitazioni/blocky-the-blockchain/node_modules/@nomicfoundation/hardhat-ethers/src/internal/signers/signers.ts:155:12)
    at async staticCallResult (/home/gabs/Documenti/Università/Software Security & Blockchain/Esercitazioni/blocky-the-blockchain/node_modules/ethers/src.ts/contract/contract.ts:337:22)
    at async staticCall (/home/gabs/Documenti/Università/Software Security & Blockchain/Esercitazioni/blocky-the-blockchain/node_modules/ethers/src.ts/contract/contract.ts:303:24)
    at async Proxy.getTicket (/home/gabs/Documenti/Università/Software Security & Blockchain/Esercitazioni/blocky-the-blockchain/node_modules/ethers/src.ts/contract/contract.ts:351:41)
    at async REPL100:1:33 {
  code: 3,
  data: '0x08c379a000000000000000000000000000000000000000000000000000000000000000200000000000000000000000000000000000000000000000000000000000000019746963636b61206d7573742062652076616c6964206d6174652100000000000000'
}
> 
```

Blockchain out:

```
eth_chainId
eth_accounts
eth_blockNumber
eth_getBlockByNumber
eth_feeHistory
eth_maxPriorityFeePerGas
eth_estimateGas
eth_sendTransaction
  Contract deployment: Ticket
  Contract address:    0x5fbdb2315678afecb367f032d93f642f64180aa3
  Transaction:         0xdcc20a0b58ecb46212f98970ad0087ccc0b8413c5f00e5c3df826cd1065c7ce1
  From:                0xf39fd6e51aad88f6f4ce6ab8827279cfffb92266
  Value:               0 ETH
  Gas used:            570706 of 570706
  Block #1:            0x11fdb703be33deb62adc5eb5b4282cdf2f9f72be91dab99224d713da37df03ae

eth_getTransactionByHash
eth_getTransactionReceipt
eth_blockNumber
eth_feeHistory
eth_estimateGas
eth_sendTransaction
  Contract call:       Ticket#initTicket
  Transaction:         0xc1da6cae2ab4d3fb2d3dd76a3fab7cdcafb322abaaa59cfee18a627dc2963ca5
  From:                0xf39fd6e51aad88f6f4ce6ab8827279cfffb92266
  To:                  0x5fbdb2315678afecb367f032d93f642f64180aa3
  Value:               0 ETH
  Gas used:            111427 of 111427
  Block #2:            0xd83bcf423150c682146ceb3fcd56e3c70871bb21416a6a8cc3981601c593ccfd

eth_getTransactionByHash
eth_getTransactionReceipt
eth_blockNumber
eth_feeHistory
eth_estimateGas
  Contract call:       Ticket#initTicket
  From:                0xf39fd6e51aad88f6f4ce6ab8827279cfffb92266
  To:                  0x5fbdb2315678afecb367f032d93f642f64180aa3
  Value:               0 ETH

  Error: reverted with reason string 'Ticket already exist mate!'

eth_accounts
eth_blockNumber
eth_feeHistory
eth_estimateGas
eth_sendTransaction
  Contract call:       Ticket#invalidateTicket
  Transaction:         0x5b4a5e24251735813480cabfe63185ae063f7ea959113e9a6023850185db5659
  From:                0xf39fd6e51aad88f6f4ce6ab8827279cfffb92266
  To:                  0x5fbdb2315678afecb367f032d93f642f64180aa3
  Value:               0 ETH
  Gas used:            24692 of 29515
  Block #3:            0x9589a14ec47196494b29ddd13eb85951501a15ee59c54e206b1c020367ca763c

eth_getTransactionByHash
eth_getTransactionReceipt
eth_call
  Contract call:       Ticket#getTicket
  From:                0xf39fd6e51aad88f6f4ce6ab8827279cfffb92266
  To:                  0x5fbdb2315678afecb367f032d93f642f64180aa3

  Error: reverted with reason string 'ticka must be valid mate!'
```

2. Create a nested structure to manage an online shopping cart, where each item includes its name, price, and a nested shipping address.

Code:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

// NOTE: a possible improvement could be the use of "indexed" keywords to allow off-chain filtering.


contract ShoppingCart {

    struct ShippingAddress {
        string street;
        string city;
        string zipCode;
    }

    // Item struct that includes the name, price, and shipping address of the item
    struct Item {
        string name;
        uint256 price;
        ShippingAddress shippingAddress;
    }

    // Mapping to store the shopping cart for each address
    mapping(address => Item[]) private carts;

    // Adds an item to the sender's shopping cart
    function addItem(
        string memory name,
        uint256 price,
        string memory street,
        string memory city,
        string memory zipCode
    ) public {
        ShippingAddress memory addr = ShippingAddress(street, city, zipCode);
        carts[msg.sender].push(Item(name, price, addr));
    }

    // Returns entire shopping cart for the sender
    function getCart() public view returns (Item[] memory) {
        return carts[msg.sender];
    }

    // Returns a specific item from the sender's shopping cart by index
    function getItem(uint256 index) public view returns (Item memory) {
        require(index < carts[msg.sender].length, "Indice non valido");
        return carts[msg.sender][index];
    }

    // Returns total price
    function getTotalPrice() public view returns (uint256) {
        uint256 total = 0;
        for (uint256 i = 0; i < carts[msg.sender].length; i++) {
            total += carts[msg.sender][i].price;
        }
        return total;
    }

    // Empties sender's shopping cart
    function clearCart() public {
        delete carts[msg.sender];
    }
}
```

Results:

```
> const { ethers } = await network.connect()
undefined
> const cart = await ethers.deployContract("ShoppingCart");
undefined
> await cart.waitForDeployment()
Proxy [
  BaseContract {
    target: '0x5FbDB2315678afecb367f032d93F642f64180aa3',
    interface: Interface {
      fragments: [Array],
      deploy: [ConstructorFragment],
      fallback: null,
      receive: false
    },
    runner: HardhatEthersSigner {
      address: '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266',
      provider: HardhatEthersProvider {}
    },
    filters: Proxy [ {}, [Object] ],
    fallback: null,
    [Symbol(_ethersInternal_contract)]: {}
  },
  { get: [Function: get], has: [Function: has] }
]
> await (await cart.addItem("NVIDIA RTX PRO 6000", 17000, "via varano 143z", "Ancona", "60131")).wait()
ContractTransactionReceipt {
  provider: HardhatEthersProvider {},
  to: '0x5FbDB2315678afecb367f032d93F642f64180aa3',
  from: '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266',
  contractAddress: null,
  hash: '0x812819b926b53e714ea7071d964b07eb38dab663b617fa4c757fe0a28b0dfba0',
  index: 0,
  blockHash: '0x06bb66ba8f4477b3407a4c16b5e67b687e76ec448f07e139846505fec47ac54b',
  blockNumber: 2,
  logsBloom: '0x00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000',
  gasUsed: 162103n,
  blobGasUsed: undefined,
  cumulativeGasUsed: 162103n,
  gasPrice: 974246998n,
  blobGasPrice: undefined,
  type: 2,
  status: 1,
  root: undefined
}
> await (await cart.addItem("AMD Threadripper 9995WX", 12000, "via varano 143z", "Ancona", "60131")).wait()
ContractTransactionReceipt {
  provider: HardhatEthersProvider {},
  to: '0x5FbDB2315678afecb367f032d93F642f64180aa3',
  from: '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266',
  contractAddress: null,
  hash: '0x3c91742c774ad0637ea10daf9e812a12ea811e27870542114ebb0cb4ec2850d6',
  index: 0,
  blockHash: '0x3886d40d4614170c87dfb50e922abd281e6af0a51da443678007a1b1d9686edf',
  blockNumber: 3,
  logsBloom: '0x00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000',
  gasUsed: 145051n,
  blobGasUsed: undefined,
  cumulativeGasUsed: 145051n,
  gasPrice: 853124159n,
  blobGasPrice: undefined,
  type: 2,
  status: 1,
  root: undefined
}
```

```
> await (await cart.addItem("DDR5 RAM", 7000, "via varano 143z", "Ancona", "60131")).wait()
ContractTransactionReceipt {
  provider: HardhatEthersProvider {},
  to: '0x5FbDB2315678afecb367f032d93F642f64180aa3',
  from: '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266',
  contractAddress: null,
  hash: '0x6c6960c9ac40370e76ceaa75e769a41acbb8aa82a2ea82d0ce6b44763559b0f3',
  index: 0,
  blockHash: '0x4a1b670fe705a7276917dc0bad243f8cb924b9a1a753cce136c5d2e865a1a412',
  blockNumber: 4,
  logsBloom: '0x00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000',
  gasUsed: 144871n,
  blobGasUsed: undefined,
  cumulativeGasUsed: 144871n,
  gasPrice: 746999250n,
  blobGasPrice: undefined,
  type: 2,
  status: 1,
  root: undefined
}
> await cart.getCart()
Proxy [
  Result(3) [
    Proxy [ [Result], [Object] ],
    Proxy [ [Result], [Object] ],
    Proxy [ [Result], [Object] ]
  ],
  { get: [Function: get] }
]
> await cart.getCart().toString()
'[object Promise]'
> (await cart.getTotalPrice()).toString()
'36000'
> await (await cart.clearCart()).wait()
ContractTransactionReceipt {
  provider: HardhatEthersProvider {},
  to: '0x5FbDB2315678afecb367f032d93F642f64180aa3',
  from: '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266',
  contractAddress: null,
  hash: '0xb984c8b17751bfc4feadfa73797334dabd3de40c8cd4791b8bca329bb3ae96d3',
  index: 0,
  blockHash: '0x7ef7863d4d3bd780a9f2f8fe3dd65aaf2c912f808538ab1a9aad408fd7106163',
  blockNumber: 5,
  logsBloom: '0x00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000',
  gasUsed: 83435n,
  blobGasUsed: undefined,
  cumulativeGasUsed: 83435n,
  gasPrice: 654075255n,
  blobGasPrice: undefined,
  type: 2,
  status: 1,
  root: undefined
}
> await cart.getCart()
Proxy [ Result(0) [], { get: [Function: get] } ]
```